

This Mathematica code is part of the lecture script *Introduction to Computational Quantum Mechanics* by Roman Schmied. It is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.



You can download the associated lecture script at <http://arxiv.org/abs/1403.7050>

many ways to define the factorial function

The following list of definitions of the factorial function is based on the Wolfram demo <http://www.wolfram.com/training/videos/EDU002/>. Try to understand as many of these definitions as possible. What this means in practice is that for most problems you can pick the programming paradigm that suits your way of thinking best, instead of being forced into one way or another. The different paradigms have different advantages and disadvantages, which may become clearer to you as you become more familiar with them.

1. function identity

Define the function f to be an alias of the built-in function Factorial: $f[5]$ is now strictly the same thing as calling Factorial[5], which in turn is the same thing as calling $5!$.

```
Clear[f];  
f = Factorial;
```

2. function forwarding

A call to f is forwarded to the function “!”: calling $f[5]$ triggers the evaluation of $5!$.

```
Clear[f];  
f[n_] := n!
```

3. mathematical equivalence (I)

Use the mathematical definition $n \neq \Gamma(n + 1)$:

```
Clear[f];  
f[n_] := Gamma[n + 1]
```

4. mathematical equivalence (II)

Use the mathematical definition $n! = \prod_{i=1}^n i$:

```
Clear[f];
f[n_] := Product[i, {i, n}]
```

5. recursion (I): pattern matching

Rule-based recursion, using Mathematica's built-in pattern-matching capabilities: calling $f[5]$ leads to a call of $f[4]$, which leads to a call of $f[3]$, and so on until $f[1]$ immediately returns the result 1, after which the program unrolls the recursion stack and does the necessary multiplications:

```
Clear[f];
f[1] = 1;
f[n_] := n f[n - 1]
```

6. recursion (II): If clause

The same recursion but without rules (no pattern-matching):

```
Clear[f];
f[n_] := If[n == 1, 1, n f[n - 1]]
```

7. recursion (III): anonymous function

Define the same recursion through functional programming: f is a function whose name is `#0` and whose first (and only) argument is `#1`. The end of the function definition is marked with `&`.

```
Clear[f];
f = If[#1 == 1, 1, #1 #0[#1 - 1]] &;
```

8. procedural programming (I): Do loop

procedural programming with a Do loop:

```
Clear[f];
f[n_] := Module[{t = 1},
  Do[t = t i, {i, n}];
  t]
```

9. procedural programming (II): For loop

procedural programming with a For loop: this is how you would compute factorials in procedural programming languages like C. It is a very precise step-by-step prescription of how exactly the computer is supposed to do the calculation.

```
Clear[f];
f[n_] := Module[{t = 1, i},
  For[i = 1, i ≤ n, i++, t *= i];
  t]
```

10. list processing (I): automatic

Make a list of the numbers $1 \dots n$ (with `Range[n]`) and then multiply them together at once, by applying the function `Times` to this list. This is the most elegant way of multiplying all these numbers together, because both the generation of the list of integers and their multiplication are done with internally optimized methods. The programmer merely specifies *what* he would like the computer to do, and not *how* it is to be done.

```
Clear[f];
f[n_] := Times @@ Range[n]
```

11. list processing (II): manual

Make a list of the numbers $1 \dots n$ and then multiply them together one after the other.

```
Clear[f];
f[n_] := Fold[Times, 1, Range[n]]
```

12. functional programming

Functional programming: make a list of functions $\{t \rightarrow t, t \rightarrow 2t, t \rightarrow 3t, \dots, t \rightarrow nt\}$, and then, starting with the number 1, apply each of these functions once.

```
Clear[f];
f[n_] := Fold[#2[#1] &, 1, Array[Function[t, #1 t] &, n]]
```

13. permutation group size

Construct a list whose length we know to be $n!$:

```
Clear[f];
f[n_] := Length[Permutations[Range[n]]]
```

14. repeated pattern matching and replacement

Use repeated pattern-based replacement (`//.`) to find the factorial: start with the object $\{1, n\}$ and apply the given rule until the result no longer changes because the pattern no longer matches.

```
Clear[f];
f[n_] := First[{1, n} //. {a_, b_ /; b > 0} -> {b a, b - 1}]
```

15. string processing

Build a string whose length is n !

```
Clear[f];
f[n_] := StringLength[Fold[StringJoin[Table[#, {#2}]] &, "A", Range[n]]]
```

16. nested lists by repeated replacement

Starting from the number n , repeatedly replace each number m by a list containing m times the number $m - 1$. At the end, we have a list of lists of ... of lists that overall contains $n!$ times the number 1. Flatten it out and count the number of elements.

```
Clear[f];
f[n_] := Length[Flatten[n //. m_ /; m > 1 -> Table[m - 1, {m}]]]
```

17. analytic differentiation

Use the relationship $\frac{d^n}{dx^n}(x^n) = n!$: compute the n^{th} derivative of x^n .

```
Clear[f];
f[n_] := D[x^n, {x, n}]
```